

Rapport des travaux

Audit, PLC Studio, skill de création de projets, nouvelles cartes, axes multiples, Modbus TCP et Ethernet, variateurs StepperOnline et robots AUBO / FAIRINO.

1	Synthèse	vue d'ensemble, chiffres clés
2	Audit initial du dépôt	5 oct. 2026
3	PLC Studio : l'outil de configuration	6 oct. 2026
4	Skill « plc-projet » et ligne de commande	7 oct. 2026
5	Nouvelles cartes automates	7 oct. 2026
6	Axes multiples	7 oct. 2026
7	Modbus TCP et Ethernet	7 oct. 2026
8	Variateurs StepperOnline	7 oct. 2026
9	Robots AUBO et FAIRINO	7 oct. 2026
10	Vérifications, limites et points à valider	
11	État du dépôt et prochaines étapes	

Rapport établi le 7 octobre 2026 à partir de l'historique Git du dépôt `alexisaeq/plc` (branches `main` et `claude/cartes-et-axes`). Les chemins cités sont relatifs à la racine du dépôt.

1 Synthèse

Le travail a transformé un firmware écrit pour une machine précise (application `RessortRoyal2` sur KinCony KC868-A8S avec un servo SureServo) en une plateforme : un outil graphique, **PLC Studio**, génère maintenant la configuration, les écrans et le code C++ d'une machine ; le firmware accepte huit cartes automates, plusieurs axes de types différents, le Modbus en RS485 ou en réseau, et peut piloter des robots.

8

cartes automates prises en charge (1 au départ)

8

types d'axes :
SureServo, Lichuan ×2,
StepperOnline ×4, pas-à-pas

68

tests automatiques de PLC Studio, tous verts

7

projets d'exemple dont le C++ généré est vérifié

Les grandes étapes

- **Audit** — 225 incohérences relevées dans le firmware et l'application existante, dont 3 critiques (chapitre 2).
- **PLC Studio** — outil web local en sept étapes : projet, équipements, câblage, grafcet, écrans, simulation, génération ; guide PDF de 16 pages (chapitre 3).
- **Skill « plc-projet »** — création d'un projet complet à partir d'une description de machine, validé par des scénarios de simulation (chapitre 4).
- **Cartes** — le firmware ne contient plus de code propre à une carte : sept nouvelles cartes ESP32 / ESP32-S3 décrites par leur fichier matériel (chapitre 5).
- **Axes multiples** — un `AxisController` commun, SureServo multi-instance, Lichuan A6/A5, pas-à-pas STEP/DIR (chapitre 6).
- **Modbus TCP et Ethernet** — chaque équipement Modbus passe par le RS485 ou par le réseau ; Ethernet des cartes qui en ont (chapitre 7).
- **StepperOnline** — pas-à-pas DM556RS / DM882RS / CL57RS / CL86RS et servos A6-RS / T6 (chapitre 8).
- **Robots** — AUBO (ARCS) et FAIRINO (série FR) en Modbus TCP, plus un équipement Modbus générique (chapitre 9).

À retenir. Rien de ce qui a été ajouté depuis le chapitre 5 n'a encore tourné sur un ESP32 ni sur du matériel réel : la compilation ESP32 a été laissée de côté à ta demande (tu la fais toi-même) et plusieurs adresses Modbus des variateurs StepperOnline et des robots sont supposées, faute d'accès aux documentations officielles complètes. Le chapitre 10 liste ce qu'il faut valider.

2 Audit initial du dépôt

Première étape : une lecture complète du firmware BorneUniverselle et de l'application RessortRoyal2 pour relever les incohérences avant de construire quoi que ce soit dessus. Résultat : `AUDIT_INCOHERENCES.md` (1 199 lignes).

225

constats après fusion
des doublons

3

critiques (risque
physique)

39

majeurs

183

mineurs (139) et
cosmétiques (44)

Constats critiques

Id	Constat
CR-1	En mode aller-retour, la cible PR1 du servo n'est jamais comparée à la butée physique : les butées logicielles sont contournables.
CR-2	La supervision du servo, l'arrêt d'urgence logiciel et l'arrêt du jog ne tournent que si un navigateur est connecté à l'automate.
CR-3	Le jog SureServo attend des appels cycliques, alors que l'application ne l'appelle qu'au front d'appui (butée et zone lente non surveillées).

Autres thèmes relevés

- **Entrées réseau fragiles** : débordement de pile possible sur une commande de suppression de fichier, validation de config.json avec effets de bord.
- **Build non reproductible** : casse d'inclues, fichiers de partitions introuvables, environnement par défaut différent de celui déployé.
- **Front et back divergents** : messages FATAL non affichés, protections de fichiers et de mots de passe incohérentes.

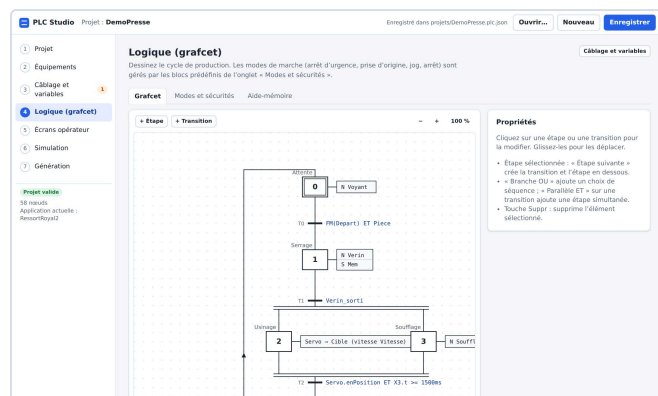
Suite donnée. L'audit est une liste de constats avec corrections proposées ; il n'a pas été corrigé en bloc. Le code généré par PLC Studio évite plusieurs de ces pièges (butées vérifiées par l'AxisController, jog cyclique, arrêt du jog à l'arrêt). En revanche **CR-2 reste ouvert** : la logique ne tourne toujours qu'avec un écran connecté (la clé `execution_mode: "headless"` est écrite par PLC Studio mais pas encore lue par le firmware).

3 PLC Studio : l'outil de configuration

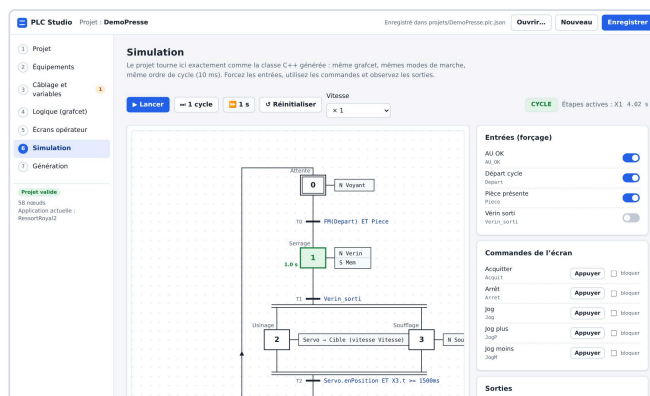
Application web locale (Node.js, sans dépendance, dans `tools/plc-studio/`) qui produit tout ce dont le firmware a besoin pour une machine, sans écrire de code à la main.

Réalisé en quatre temps

Étape	Contenu
Phase 1 — configuration	Étapes Projet, Équipements, Câblage et variables, Écrans, Génération ; génération de <code>config.json</code> , <code>interface.json</code> et du fichier de paramètres ; identifiants de nœuds calculés comme le firmware (CRC-32) ; validation des règles qui bloquent le démarrage.
Phase 2 — grafcet et C++	Langage des réceptivités (ET/OU/NON, fronts FM/FD, temps d'étape, comparaisons) ; éditeur grafcet (divergences ET/OU, actions N, S/R, affectations, compteurs, servo, messages) ; blocs « Modes et sécurités » (arrêt d'urgence + acquittement, marche/arrêt, prise d'origine, jog, couple) ; génération de la classe C++ et bascule de <code>main.cpp</code> ; installation avec sauvegarde.
Phase 3 — simulateur	Exécute le projet comme la classe C++ générée (même cycle de 10 ms, mêmes modes), avec un modèle de servo ; grafcet animé, forçage des entrées, journal. A permis de corriger deux défauts du code généré.
Relecture et écrans	Corrections issues d'une relecture du générateur et du simulateur (arrêt du jog, prise d'origine interrompue, fronts, verrou après arrêt...) ; éditeur visuel des écrans : aperçu réel de l'interface web du firmware, glisser-déposer, propriétés des widgets, mots de passe de page.



Éditeur de grafcet

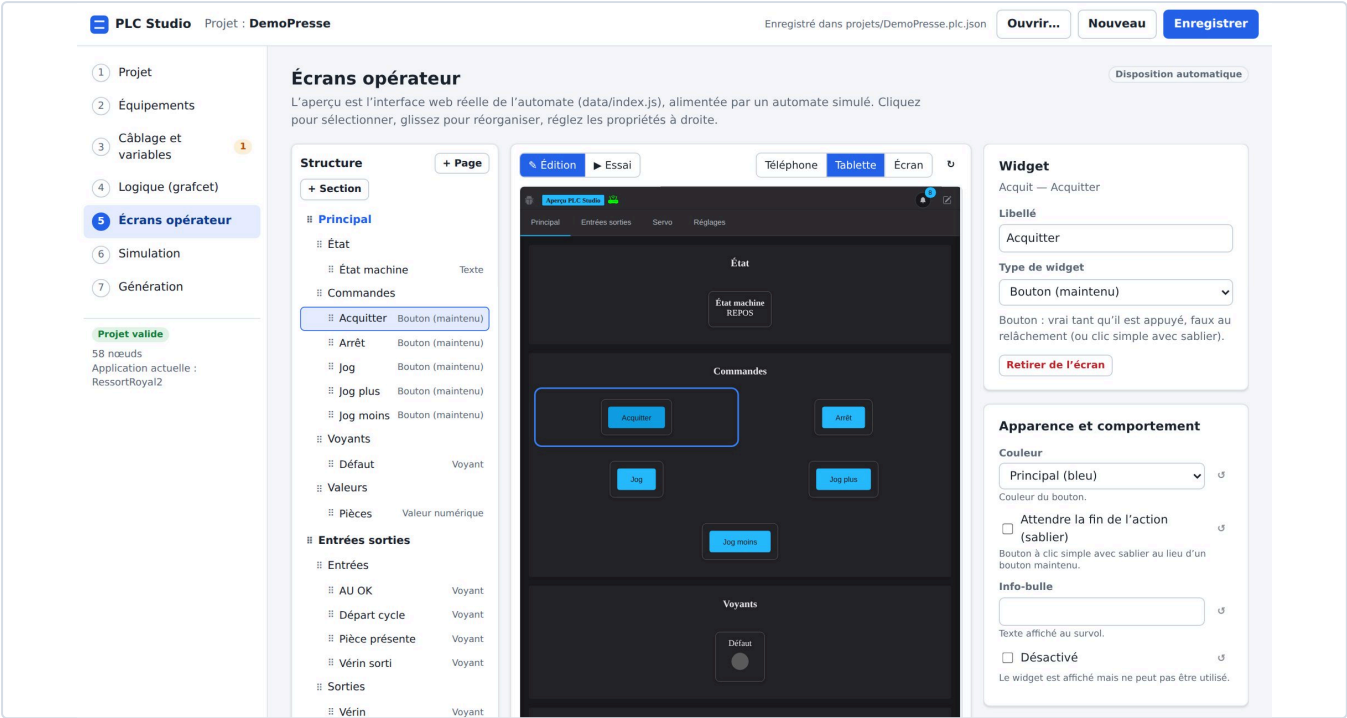


Simulateur

Fichiers produits pour une machine

Fichier	Contenu
data/config.json	réseau (WiFi, Ethernet), options, sections et nœuds
data/interface.json	pages, sections et widgets de l'écran opérateur
data/<Nom>.json	valeurs initiales des paramètres sauvegardés
data/hardware/...	variantes de carte (sans horloge, autre réglage RS485) et fichiers des équipements Modbus génériques
src/<Nom>/<Nom>.h/.cpp	classe d'application (modes de marche + grafcet + axes)
src/main.cpp	bascule vers la nouvelle classe

Documentation. Guide illustré de 16 pages, PLC_Studio_Guide.pdf à la racine du dépôt (installation, prise en main, les sept étapes, mise sur l'automate, aide-mémoire, limites).



Éditeur visuel des écrans opérateur (aperçu réel de l'interface web du firmware)

4 Skill « plc-projet » et ligne de commande

Pour créer un projet en décrivant simplement la machine (« un convoyeur qui... »), un skill Claude Code s'appuie sur un cahier des charges compact (la **spec**) converti en projet PLC Studio par le même code que l'interface.

Briques ajoutées

- `shared/spec.js` — spec → projet (voies « `plc.X1` », étapes par numéro, actions en texte court, écrans automatiques) et projet → spec, sans perte ; mise en page automatique du grafcet.
- `shared/scenario.js` — scénarios de test joués par le simulateur : `set`, `press`, `wait`, `until`, `expect`, `message`, défauts d'axe.
- `cli/plc.mjs` — commandes `catalog`, `build`, `check --cpp`, `sim`, `describe --md`, `export-spec`.
- `.claude/skills/plc-projet/` — procédure, format de la spec, sémantique de la logique et patrons, quatre exemples validés.

Essais du skill

Trois demandes réalistes ont été confiées à des agents utilisant le skill ; leurs retours ont servi à l'améliorer.

Demande	Résultat
Ensacheuse (nouvelle machine)	0 erreur, C++ vérifié, 9/9 scénarios
Table d'indexation avec servo	0 erreur, C++ vérifié, 14/14 scénarios
Modification du projet DemoPresse	0 erreur, C++ vérifié, 10/10 scénarios ; identifiants et ordre du fichier conservés

Exemples livrés : convoyeur de tri, presse servo, ensacheuse, table d'indexation.

Mis à jour depuis. Le skill connaît aussi les nouvelles cartes, les axes multiples, les variateurs StepperOnline, les robots, le Modbus TCP et l'Ethernet (chapitres 5 à 9).

5 Nouvelles cartes automates

Le firmware ne contient plus de code propre à une carte : tout ce qui la décrit (entrées/sorties, bus I2C, horloge, RS485, buzzer, Ethernet) est dans son fichier matériel

`data/hardware/<carte>.json`.

Ce que le firmware sait maintenant lire dans le fichier matériel

- Expandeurs I2C génériques : PCF8574, PCF8575, TCA9554 / PCA9554, TCA9534 / PCA9534, AW9523, voie par voie, avec ligne d'interruption.
- Horloge au choix : DS3231, PCF85063, PCF8563, RX8130, ou détection automatique ; pas d'horloge possible.
- RS485 avec broche de validation d'émission (mode semi-duplex de l'UART), vitesse et format.
- GPIO directs avec logique inversée et tirage (haut, bas, aucun) ; buzzer remis au repos à chaque connexion WiFi.
- Sorties mises à leur état de repos une seule fois, à leur création.

Cartes prises en charge

Carte	Environnement PlatformIO	E/S principales	Horloge	Ethernet
KinCony KC868-A85 (ESP32)	<code>esp32dev</code>	8 entrées, 8 relais	DS3231	—
KinCony KC868-A16 (ESP32)	<code>quatre_mb_huge</code>	16 entrées, 16 sorties MOSFET	aucune	LAN8720
KinCony KC868-A16v3 (ESP32-S3)	<code>esp32s3_n16r8</code>	16 entrées, 16 sorties MOSFET	DS3231	W5500
KinCony KC868-A8v3 (ESP32-S3)	<code>esp32s3_n16r8</code>	8 entrées, 8 relais	DS3231	W5500
Waveshare ESP32-S3-POE-ETH-8DI-8DO	<code>esp32s3_n16r8</code>	8 entrées, 8 sorties, buzzer	PCF85063	W5500 (PoE)
Waveshare ESP32-S3-RS485-WLED	<code>esp32s3_n16r8</code>	bouton BOOT, sorties 5 V GPIO2/16	aucune	—
M5Stack StamPLC (ESP32-S3)	<code>m5stack_stamplc</code>	8 entrées, 4 relais, buzzer	RX8130	—
Homemaster MiniPLC (ESP32)	<code>quatre_mb_huge</code>	4 entrées, 4 boutons, 6 relais, 2 LED	PCF8563	LAN8720

Dans PLC Studio

- Toutes les cartes au catalogue, avec l'environnement PlatformIO à compiler.
- Variantes générées du fichier matériel quand l'horloge est décochée ou que la vitesse / le format RS485 change (ex. `Waveshare_ESP32S3_POE_8DI8DO_19200_8N1`).

Brochages. Les brochages viennent des documentations publiques des fabricants ; celui de la Waveshare ESP32-S3-RS485-WLED est à confirmer sur le schéma.

6 Axes multiples

Un projet peut piloter plusieurs axes de types différents. Chacun est confié à un `AxisController` commun (`src/AxisController`) qui gère butées, vitesses, prise d'origine, jog et couple, quel que soit le variateur.

Firmware

Type d'axe	Pilote	Pilotage
SureServo 2	<code>SureServo</code> (multi-instance)	Modbus
Lichuan A6 / A5	<code>LichuanServo</code> (deux séries)	Modbus
Pas-à-pas STEP/DIR	<code>StepperMotor</code> (FastAccelStepper, moteur partagé)	GPIO directs ; ENABLE, origine, fins de course, alarme facultatifs
StepperOnline (chapitre 8)	<code>StepperOnlineRS</code> , <code>StepperOnlineServo</code>	Modbus

PLC Studio

- Axes nommés dans la logique : `Axe_X.enPosition` , `Axe_X.position` , `SERVO_MOVE[axe_x]` Cible @ `Vitesse` .
- Modes par axe : ordre des prises d'origine, jog, couple.
- Signaux des axes (servo ON, STEP, DIR, ENABLE, capteur d'origine...) câblés sur les voies de l'automate ; STEP et DIR limités aux GPIO directs.
- Un panneau par axe en simulation ; migration automatique des anciens projets à un seul servo.

Documentation : `docs/axes/lichuan.md` , `docs/axes/stepper.md` , `docs/axes/stepperonline-*.md` .

7 Modbus TCP et Ethernet

Chaque équipement Modbus (module d'E/S, variateur, robot) passe au choix par le bus RS485 de l'automate (**Modbus RTU**) ou par le réseau WiFi / Ethernet (**Modbus TCP**). Les deux se mélangent dans un même projet.

Firmware

- **MyModbus** aiguille chaque requête : RTU, ou TCP vers l'équipement enregistré (IP, port, n° d'unité). Les nœuds et les pilotes d'axes ne changent pas.
- Section **config.json** avec un bloc `"tcp": { "ip", "port" }` ; l'adresse devient le n° d'unité (passerelles TCP → RTU acceptées).
- Reconnexion au plus toutes les 3 s, délais de 500 ms ; équipement injoignable signalé comme un esclave RTU.
- **EthernetManager** : W5500 (SPI) et LAN8720 (RMII), en plus du WiFi ; serveur web et mDNS aussi sur l'Ethernet.
- Redémarrage automatique quand l'enregistrement de **config.json** change les sections ou l'Ethernet (auparavant non détecté).

PLC Studio

- Choix « RS485 (Modbus RTU) » / « Réseau (Modbus TCP) » sur chaque équipement Modbus, avec IP et port.
- **Équipement Modbus générique** : 16 bobines écrites et lues, 16 registres écrits, lus et d'entrée, à partir d'adresses réglables (notation « 40001 » acceptée) ; fichier matériel généré.
- Registres écrits utilisables comme sorties entières dans le grafcet ; carte Ethernet à l'étape Projet ; règles de validation (IP, unités, une seule connexion par IP, 8 IP et 16 équipements au plus).

The screenshot shows the 'Équipements' configuration screen in PLC Studio. The interface is in French. On the left, there is a sidebar with a list of steps: 1. Projet, 2. Équipements (selected), 3. Câblage et variables, 4. Logique (grafcet), 5. Écrans opérateur, 6. Simulation, 7. Génération. Below this, it says 'Projet valide' and '21 neurons'. The main area is titled 'Équipements' and contains a warning box: 'Attention: Modbus TCP passe par le réseau : ne l'utilisez pas pour une fonction de sécurité (arrêt d'urgence câblé obligatoire)'. Below the warning, there is a section 'Équipements du projet' with a note: 'L'automate est toujours déclaré en premier dans config.json (il porte le bus RS485). Chaque équipement Modbus passe par le RS485 (RTU) ou par le réseau WiFi / Ethernet (TCP)'. The 'Automate' section is for configuring the main PLC unit. It has a 'Nom' field with 'Automate' entered, a 'N° d'unité Modbus' field with '1' entered, and a 'Port (TCP)' field with '502' entered. The 'Robot' section is for configuring a generic Modbus device. It has a 'Nom' field with 'Robot' entered, a 'N° d'unité Modbus' field with '1' entered, and a 'Port (TCP)' field with '502' entered. The 'Liaison Modbus' dropdown menu is set to 'Réseau WiFi / Ethernet (Modbus TCP)'. The 'Adresse de Q1 (bobines écrites)' field is set to '1', and the 'Adresse de I1 (bobines lues)' field is set to '101'. The 'Adresse de W1 (registres écrits)' field is set to '40001', and the 'Adresse de R1 (registres lus)' field is set to '40101'. The 'Adresse de IR1 (registres d'entrée)' field is set to '0'. At the bottom, there is a note: 'Bobines écrites (FC05) : 16 - Bobines lues (FC01) : 16 - Registres écrits (FC06) : 16 - Registres lus (FC03) : 16 - Registres d'entrée lus (FC04) : 16 - Fichier matériel généré : data\hardware\Modbus_Robot.json'.

Équipement générique « Robot » en Modbus TCP dans PLC Studio

8 Variateurs StepperOnline

Deux familles de variateurs StepperOnline pilotés en Modbus RTU (ou via une passerelle TCP), intégrés comme axes.

Modèles	Pilote	Principe	Registres
Pas-à-pas DM556RS, DM882RS (boucle ouverte), CL57RS, CL86RS (boucle fermée)	StepperOnlineRS	Protocole Leadshine « PR » : trajet 0 écrit (cible, vitesse, rampes) puis déclenché ; origine, arrêt rapide, acquittement d'alarme. Positions en pas, vitesses en tr/min.	en partie vérifiés mot d'état et position supposés
Servo A6-RS	StepperOnlineServo	Paramètres <code>Cgg.nn</code> ; pilotage par la logique des entrées DI (servo ON, départ, origine, acquittement).	plusieurs supposés
Servo T6 RS485	StepperOnlineServo	Variateur Leadshine renommé, mode PR ; servo ON par <code>Pr4.00</code> .	plusieurs supposés

- Ces variateurs rangent les nombres 32 bits dans l'ordre inverse des nœuds du firmware : le pilote remet les mots dans l'ordre, et l'écran affiche une position recopiée de l' `AxisController` .
- A6-RS et T6 ne sont pas compatibles avec les Lichuan déjà pris en charge, d'où un pilote distinct.
- Couple : réglage du courant crête pour les pas-à-pas, limite de couple pour le T6, non disponible sur l'A6-RS.

Mise en service. Les sites des fabricants étaient inaccessibles : une partie des adresses vient de manuels de modèles voisins. Avant la première mise en mouvement, lire chaque registre une fois avec un outil Modbus. Un registre supposé qui n'existe pas fait répondre le variateur en erreur, et l'automate peut alors le considérer hors ligne : retirer ce registre du projet. Listes de contrôle : `docs/axes/stepperonline-rs.md` et `docs/axes/stepperonline-servo.md` .

9 Robots AUBO et FAIRINO

L'automate, client Modbus TCP, commande les cobots par une « poignée de main » avec un programme robot : n° de programme, départ, en cours, fini, défaut, battements de vie.

	AUBO (contrôleur ARCS)	FAIRINO (série FR)
Serveur Modbus	Port 502, unité 1, à activer dans Configuration > Bus de terrain	192.168.58.2 (IP d'usine), port 502, logiciel V3.7.3 ou plus
Échanges	Registres généraux 300 à 331 selon une convention du projet : consignes 300-308, retours 310-319, positions 320-331	Bobine 502 = départ du programme ; sorties DO0-DO7 ; signaux utilisateur pour la poignée de main ; registres de consignes et de retours
Vérifié	Plage 300-525 en registres de maintien, adressage à partir de 0, fonctions des E/S du contrôleur	Bobines 300-599, 300 = DO0, 502 = départ ; plage d'état 310-473
Supposé	Écriture FC06, unité 1, IP d'usine	Adresses des signaux utilisateur (à partir de 100), DO1-DO7 contigus, pause / reprise / arrêt (laissés désactivés)
Câblé en TOR	Départ, arrêt, pause et états critiques (procédures dans <code>docs/robots/aubo.md</code> et <code>docs/robots/fairino.md</code>)	

Ajouts associés

- Positions des robots en entiers signés de 16 bits (0,1 mm / 0,1°), recopiées par le programme robot : les flottants ne sont pas lus.
- État de repos des sorties Modbus écrit au démarrage de l'automate : un « départ » resté à 1 avant un redémarrage n'empêche plus le départ suivant.
- L'équipement Modbus générique (chapitre 7) couvre tout autre robot ou automate tiers.

Sécurité. Le Modbus TCP passe par le réseau et n'est pas une fonction de sécurité. L'arrêt d'urgence du robot et de la machine reste câblé (chaîne de sécurité) ; PLC Studio l'affiche en avertissement dès qu'un équipement est en TCP.

10 Vérifications, limites et points à valider

Ce qui a été vérifié

Vérification	Résultat
Tests automatiques de PLC Studio (<code>npm test</code>)	68 / 68 (47 après l'éditeur d'écrans, 53 après le skill)
Syntaxe du C++ généré (<code>npm run check:cpp</code>)	7 projets d'exemple OK : presse avec / sans servo, noms piégeux, axes multiples, Lichuan, cellule robot Modbus TCP, îlot AUBO / FAIRINO + StepperOnline
Syntaxe des nouveaux modules firmware	<code>MyModbus</code> , <code>EthernetManager</code> , <code>StepperOnlineRS</code> , <code>StepperOnlineServo</code> : sans erreur (g++ sur PC, en-têtes Arduino simplifiés)
Interface de PLC Studio	Pages parcourues dans un navigateur sans écran : aucune erreur JavaScript
Fichiers matériels	Chaque identifiant généré existe dans le fichier matériel correspondant (test automatique)

Ce qui reste à valider sur matériel

1. **Compilation ESP32** de chaque environnement (`esp32dev` , `quatre_mb_huge` , `esp32s3_n16r8` , `m5stack_stamp1c`) — à ta charge, comme convenu.
2. Démarrage de chaque carte : expandeurs, horloge, RS485, buzzer, brochage de la Waveshare RS485-WLED.
3. Modbus TCP : connexion, reconnexion après coupure, débit des rafraîchissements ; Ethernet W5500 et LAN8720.
4. Axes Lichuan, pas-à-pas et StepperOnline : registres supposés, sens, unités, prise d'origine.
5. Robots : adresses FAIRINO des signaux utilisateur, table officielle AUBO, programme robot de la poignée de main.

Limites connues

- Logique exécutée seulement avec un écran connecté (audit CR-2, mode headless non lu par le firmware).
- Registres 16 bits seulement pour l'équipement générique et les robots ; pas de flottants ; une transaction par registre.
- Ethernet seulement avec Arduino-ESP32 3.x (environnements `esp32s3_n16r8` et `quatre_mb_huge`).
- Modbus TCP : une seule connexion par adresse IP, 8 adresses et 16 équipements au plus ; une tentative de connexion vers un équipement absent peut bloquer la boucle jusqu'à 500 ms.
- Simulateur : modèles simplifiés des variateurs (vitesses et temps indicatifs).

Incident signalé en cours de route. Lors d'un essai de compilation ESP32 (phase arrêtée ensuite à ta demande), l'outil de téléchargement d'ESP-IDF a fait une fois un repli sans vérification TLS ; l'empreinte du fichier a été contrôlée et une protection a été ajoutée.

11 État du dépôt et prochaines étapes

La branche `main` contient l'audit, PLC Studio, le guide PDF et le skill « plc-projet » (chapitres 2 à 4). Les travaux suivants (chapitres 5 à 9) sont sur la branche `claude/cartes-et-axes`, poussée mais **pas encore fusionnée** :

Commit	Contenu
1565028	Cartes ESP32 / ESP32-S3 et axes multiples
066496a	Carte Waveshare ESP32-S3-RS485-WLED, axes Lichuan et pas-à-pas dans PLC Studio
da072c2	Modbus TCP et Ethernet : firmware et PLC Studio
e0a2d5a	Libellés du choix de liaison Modbus
4ba9f08	Pas-à-pas StepperOnline RS et robots AUBO / FAIRINO
26e76ec	Servos StepperOnline A6-RS et T6 RS485

Depuis `main` : 88 fichiers modifiés, environ 18 100 lignes ajoutées et 3 300 retirées.

Où trouver quoi

Élément	Emplacement
Rapport d'audit	<code>AUDIT_INCOHERENCES.md</code>
PLC Studio (outil, tests, CLI)	<code>tools/plc-studio/</code> — lancement : <code>npm start</code> ; tests : <code>npm test</code> , <code>npm run check:cpp</code>
Guide d'utilisation de PLC Studio	<code>PLC_Studio_Guide.pdf</code> (racine du dépôt)
Skill de création de projets	<code>.claude/skills/plc-projet/</code>
Cartes, Modbus TCP, axes, robots	<code>docs/cartes.md</code> , <code>docs/modbus-tcp.md</code> , <code>docs/axes/</code> , <code>docs/robots/</code>
Fichiers matériels	<code>data/hardware/</code> (20 fichiers)
Projets PLC Studio	<code>projets/</code> (ex. <code>DemoPresse.plc.json</code>)

Prochaines étapes proposées

1. Compiler les quatre environnements PlatformIO et me transmettre les erreurs éventuelles.
2. Essayer une carte et un équipement Modbus TCP (par exemple un module Waveshare derrière une passerelle) avant les robots.
3. Mettre en service les variateurs StepperOnline avec les listes de contrôle de `docs/axes/` et corriger les registres supposés.
4. Obtenir la table Modbus officielle AUBO et l'annexe des adresses FAIRINO, puis compléter les fichiers matériels.
5. Ouvrir une demande de fusion de `claude/cartes-et-axes` vers `main` quand les essais sont concluants.
6. Traiter le constat CR-2 de l'audit (logique sans écran connecté) si l'automate doit tourner sans supervision.